6/11/2025

TIME COMPLEXITY:

THE EFFICENCY OF ALGORITHM IS IMPORTANT IN COMPETIVE PROGRAMMING. THE TIME COMPLEXITY OF AN ALGORITHM ESTIMATES HOW MUCH TIME IT WILL USE FOR SOME INPUT. THE IDEA IS TO REPRESENT THE EFFICENCY AS A FUNCTION WHOSE PARAMETER IS THE SIZE OF THE INPUT.

CALCULATION RULES:

THE TIME COMPLEXITY OF AN ALGORITHM IS DENOTED $O(\ldots)$ WHERE THE "..." IS SOME FUNCTION. USUALLY THE VARIABLE n DENOTES THE INPUT SIZE. FOR EXAMPLE, IF THE INPUT IS AN ARRAY OF NUMBERS, n WILL BE THE SIZE OF THE ARRAY.

LOOPS:

IF THERE ARE K NESTED LOOPS, THE TIME COMPLEXITY IS $O(n^K)$.

FOR EXAMPLE:

```
for (int i = 0; i <= n; i++) {
        // code
}
```

$O(n)$

```
for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= n; j++) {
                // code
        }
}
```

$O(n^2)$

ORDER OF MAGNITUDE:

A TIME COMPLEXITY, DOES NOT TELL US THE EXACT NUMBER OF TIMES THE CODE INSIDE A LOOP IS EXECUTED, BUT ONLY SHOWS THE ORDER OF MAGNITUDE.

Ex:

```
for (int i = 0; i <= 3*n; i++) {
        // code
}
```

$3n$ TIMES

```
for (int i = 0; i <= 5+n; i++) {
        // code
}
```

$(5+n)$ TIMES

```
for (int i = 0; i <= n; i+=2) {
        // code
}
```

$[n/2]$ TIMES

$O(n)$

PHASES:

THE TOTAL TIME COMPLEXITY IS THE LARGEST TIME COMPLEXITY OF A SINGLE PHASE:

```
for (int j = 0; j <= n; j++) {
        // code
}
for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= n; j++) {
                // code
        }
}
for (int j = 0; j <= n; j++) {
        // code
}
```

→ $O(n)$

→ $O(n^2)$ → THIS IS THE TOTAL TIME COMPLEXITY.

→ $O(n)$

SEVERAL VARIABLES:

THIS TIME COMPLEXITY FORMULA CONTAINS SEVERAL VARIABLES:

```
for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
                // code
        }
}
```

→ $O(n \cdot m)$

RECURSION:

THE TIME COMPLEXITY OF A RECURSIVE FUNCTION DEPENDS ON THE NUMBER OF TIMES THE FUNCTION IS CALLED.

EX:

```
void f(int n) {
        if(n == 1) return;
        f(n - 1);
}
```

→ THE CALL $f(n)$ CAUSES n FUNCTION CALLS, AND THE TIME COMP. OF EACH IS $O(1)$, THUS, THE TOTAL TIME COMPLEXITY IS $O(n)$.

```
void f(int n) {
        if(n == 1) return;
        f(n - 1);
        f(n - 2);
}
```

→ EACH FUNCTION CALL CAN GENERATE 2 OTHER CALLS, EXEPT FOR $n = 1$

$$1 + 2 + 4 + \ldots + 2^{n-1} = 2^n - 1 = O(2^n)$$

COMPLEXITY CLASSES:

- $O(1) \rightarrow$ DIRECT FORMULA
- $O(\log n)$
- $O(\sqrt{n})$
- $O(n) \rightarrow$ LINEAR (ALMOST THE BEST)
- $O(n \cdot \log n) \rightarrow$ SORTING
- $O(n^2) \rightarrow$ QUADRATIC
- $O(n^3) \rightarrow$ CUBIC
- $O(2^n) \rightarrow$ OFTEN INDICATES THE PRESENCE OF SUBSET : $\{1,2,3\} = \emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{2,3\}, \{1,3\}, \{1,2,3\}$.
- $O(n!)$ → PERMUTATIONS OF THE INPUT ELEMENTS

$$(1,2,3)! = (1,2,3), (1,3,2), (2,1,3), (2,3,1), (3,2,1), (3,1,2)$$

INPUT SIZE	REQUIRED TIME COMPLEXITY
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$ OR $O(n)$
n IS LARGE	$O(1)$ OR $O(\log n)$

MAXIMUM SUBARRAY SUM:

THERE ARE OFTEN SEVERAL POSSIBLE ALGORITHM FOR SOLVING A PROBLEM SUCH THAT THEIR TIME COMPLEXITY ARE DIFFERENT.

GIVEN AN ARRAY OF n NUMBERS, OUR TASK IS TO CALCULATE THE MAXIMUM SUBARRAY SUM, THE LARGEST POSSIBLE SUM OF SEQUENCE VALUES IN THE ARRAY. FOR EXAMPLE:

-1	2	4	-3	5	2	-5	2

THE FOLLOWING SUBARRAY PRODUCES THE MAXIMUM SUM 10:

-1	2	4	-3	5	2	-5	2	$= 2+4-3+5+2 = \underline{\underline{10}}$

WE ASSUME THAT AN EMPTY SUBARRAY EXISTS, SO THE MAXIMUM SUBARRAY IS ALWAYS AT LEAST 0.

ALGORITHM 1:

A STRAIGHTFORWARD WAY TO SOLVE THE PROBLEM IS TO THROUGH ALL POSSIBLE SUBARRAYS, CALCULATE THE SUM OF VALUES IN EACH SUBARRAY AND MAINTAIN THE MAXIMUM SUM.

```
int best = 0;
for (int a = 0; a < n; a++) {
    for (int b = a; b < n; b++) {
        int sum = 0;
        for (int k = a; k <= b; k++) {
            sum += arr[k];
        }
        best = max(best, sum);
    }
}
cout << best << "\n";
```

TIME COMPLEXITY: $O(n^3)$

ALGORITHM 2:

IT IS EASY TO MAKE ALGORITHM 1 MORE EFFICIENT BY REMOVING ONE LOOP FROM IT. THIS IS POSSIBLE BY CALCULATING THE SUM AT THE SAME TIME WHEN THE RIGHT END OF THE ARRAY MOVES:

```
int best = 0;
for (int a = 0; a < n; a++) {
    for (int b = a; b < n; b++) {
        int sum = 0;
        sum += arr[b];
        best = max(best, sum);
    }
}
cout << best << "\n";
```

TIME COMPLEXITY: $O(n^2)$

ALGORITHM 3:

```
int best = 0, sum = 0;
for (int a = 0; a < n; a++) {
    sum = max(sum + arr[a], arr[a]);
    best = max(best, sum);
}
cout << best << "\n";
```

THE IDEA IS TO CALCULATE FOR EACH POSITION, THE MAXIMUM SUM OF A SUBARRAY THAT ENDS AT THAT POSITION.

SUM = MAX(2, 2+4) = 6
BEST = MAX(2, 6) = 6

0 a-1

~~-1~~	2	4	-3	5	2	-5	2

SUM = 0 BEST = 0

SUM = MAX(-1, 0+2) = 2
BEST = MAX(0, 2) = 2

SUM = MAX(10-5, -5) = 5
BEST = MAX(10, 5) = 10